

Bellcourt Review Copilot: README and setup

Runnable solution. Live: <https://bellcourt-review-copilot.vercel.app> Code: <https://github.com/devaramanelohith/bellcourt-review-copilot>

What it is

An Agentic RAG workspace for prior authorization review that runs beside PACE. A case is raised by form, case file, fax image or PDF. Code checks completeness, eligibility and the clock, and selects the governing rules by client, service and date. A model reads those sections and checks each criterion with quoted evidence. A nurse or physician decides, and a decision letter is generated from the decision. Sign-in is required and roles are enforced on the server.

Try the live demo

- Open <https://bellcourt-review-copilot.vercel.app> and sign in with a demo login from the submission comment (nurse, physician, intake or auditor).
- **Queue:** a few sample cases sorted by priority. Open PA-2609-8100 to see a recommendation with criteria, quoted evidence, and the retired policy version and memo listed as excluded.
- **New case:** upload a fax image (samples in data/fax/) or an open-case file (samples in data/open_cases/), then run the review.
- **Decide:** approve as nurse, or sign in as physician to deny. Open the letter under Decision letters and download the PDF.
- **Evidence** and **Audit log** pages show the test results and every recorded action.

Run it locally

```
git clone https://github.com/devaramanelohith/bellcourt-review-copilot
cd bellcourt-review-copilot
cp .env.example .env          # set AUTH_SECRET; add OPENROUTER_API_KEY for live reviews
python3 scripts/set_password.py nurse nurse "Your Name" # create a login
python3 scripts/dev_server.py # Python 3.10+, no packages needed
# open http://localhost:8765
```

The runtime uses only the Python standard library. Without a model key the app still shows all saved reviews.

Reproduce the evidence

```
python3 -m venv .venv && .venv/bin/pip install pdfplumber pytest reportlab pypdfium2 pypdf matplotlib
.venv/bin/python -m pytest -q # 36 tests, no key needed
.venv/bin/python scripts/ingest.py # PDFs -> sections, registry, vectors
.venv/bin/python scripts/run_open.py # 30 open cases
.venv/bin/python scripts/run_eval.py # 120 audited cases + adversarial tests
.venv/bin/python scripts/build_docs.py # regenerate every PDF from the results
```

Deploy on Vercel

```
vercel login
vercel env add AUTH_SECRET production # any long random string
vercel env add OPENROUTER_API_KEY production # for live reviews
vercel env add SUPABASE_URL production # optional: persistence
vercel env add SUPABASE_SERVICE_KEY production
vercel env add GMAIL_USER production # optional: email letters
vercel env add GMAIL_APP_PASSWORD production
vercel env add LETTER_TO_EMAIL production
vercel --prod --yes
```

Supabase needs one table; the SQL is in docs/SETUP_DATABASE_AND_EMAIL.md.

How the code is organised

File	Role
copilot/core.py	Deterministic core: registry, rule resolver, read and search tools, eligibility, clock, de-identification, verifier
copilot/agent.py	Prompts and the bounded agent loop: plan, read, review, verify, second check. Fax and text reading
copilot/pipeline.py	run_case(): ties the steps together and applies overrides the model cannot undo
copilot/cases.py, letters.py, store.py, auth.py	Case lifecycle and decisions; letters, PDF and email; Supabase or file storage; sign-in and roles
copilot/llm.py	The only module that calls a model (OpenRouter or Gemini API)
api/*.py	Serverless endpoints: login, cases, data, review, ask
public/index.html	The workspace: queue, new case, case review, decision letters, policy library, evidence, audit log
scripts/, tests/, data/	Ingestion, evaluation, document builder; 36 tests; knowledge base, sample cases, saved results